

FixFlow

Smart Maintenance Reporting Platform for Corridor Hill Residence

Product & Engineering Documentation

SRS • Data Model • System Architecture • Security • Delivery Plan

Document Version: 1.0

Date: 16 September 2026

Status: Baseline for stakeholder review and build sign-off

Prepared for: Corridor Hill Residence — Student Innovation Challenge

Table of Contents

1. Executive Summary.....	3
2. Problem Statement.....	3
3. Project Objectives.....	4
4. Project Scope.....	4
5. Stakeholders.....	5
6. User Roles & Access Model.....	5
7. Verified Student Access & Onboarding.....	6
8. Functional Requirements.....	7
9. Non-Functional Requirements.....	7
10. User Stories.....	8
11. Maintenance Workflow (State Machine).....	8
12. Use Cases.....	9
13. Differentiating Features.....	10
14. Database Schema.....	11
15. System Architecture.....	17
16. API Design.....	18
17. Security & Privacy.....	18
18. Notification & Communication Layer.....	19
19. Dashboard Design.....	19
20. Requirements Traceability.....	20
21. Acceptance Criteria.....	20
22. Testing Strategy.....	21
23. Risk Register.....	21
24. Analytics & Success Metrics.....	22
25. SDLC Roadmap.....	22
26. MVP Delivery Sequence.....	22
27. Final Deliverables Checklist.....	23
28. Conclusion.....	23

1. Executive Summary

FixFlow is the product name for the Corridor Hill Residence Smart Maintenance Reporting System: a secure, residence-specific web platform that replaces fragmented, informal maintenance reporting with one accountable digital workflow connecting verified students, maintenance staff and residence administrators.

FixFlow is built around a complete operational lifecycle rather than a simple complaint form:

verify → report → classify → prioritise → assign → work → verify resolution → close → learn

The product is designed to be simple enough for a student to report a leaking tap in under a minute, while giving maintenance teams and administrators the operational visibility needed to reduce lost requests, duplicated work, unclear ownership and unresolved issues.

1.1 Design Principle

Simple for students. Operational for maintenance. Intelligent for administrators. Secure by default.

1.2 MVP Success Condition

A real maintenance request can enter FixFlow and be traceable end-to-end: to a responsible staff member, a documented resolution action, and a final closure state confirmed by the student who reported it.

1.3 What This Document Covers

- The full Software Requirements Specification (functional, non-functional, user stories, use cases).
- The relational database schema and entity relationships.
- The system architecture, authentication design, API design and security architecture.
- Dashboard design for each role.
- Requirements traceability and acceptance criteria.
- The testing strategy, risk register and MVP delivery roadmap.

2. Problem Statement

Corridor Hill Residence currently has no single, reliable channel for reporting and resolving maintenance issues. This creates operational and experience problems for every stakeholder group:

- Maintenance issues are reported through channels that are difficult to track consistently.
- Requests often lack sufficient information — photos, location detail or urgency context.
- Students have no visibility into whether an issue has been received, assigned or resolved.
- Maintenance teams work from scattered messages or paper-based requests instead of a structured queue.
- Residence management has no measurable evidence of response times, recurring faults, workload or unresolved issues.

3. Project Objectives

3.1 Primary Objective

Build a secure, residence-specific digital maintenance platform (FixFlow) that enables authorised Corridor Hill students to report issues, and enables maintenance staff and administrators to manage those issues from submission through verified resolution.

3.2 Supporting Objectives

- Provide verified student access.
- Allow students to report maintenance problems with structured detail and evidence.
- Generate a unique reference number for every request.
- Allow students to track requests through a transparent status timeline.
- Allow maintenance staff to receive and manage assigned work.
- Allow administrators to assign and monitor work across the residence.
- Maintain a complete, auditable status history for every request.
- Provide timely, role-appropriate notifications.
- Maintain a full audit trail of sensitive actions.
- Provide operational information and analytics to residence management.
- Protect user and residence information at every layer.
- Support future expansion to additional residences without a rewrite.

3.3 Innovation Goal

Differentiation should come from the workflow and intelligence around maintenance, not from adding unrelated features. Every enhancement FixFlow adds should reduce time, improve data quality, increase accountability, or improve safety and security.

4. Project Scope

The first release focuses on one residence and one high-value problem: maintenance issue management. The architecture is extensible, but the MVP is deliberately bounded to what is achievable and demonstrable.

4.1 In Scope for MVP

- Verified Corridor Hill student access
- First-time password setup and secure login
- Maintenance issue submission (category, location, description, evidence)
- Maintenance categories and priorities
- Photo / evidence upload
- Request reference numbers
- Request status tracking and student notifications
- Maintenance staff queue and work assignment
- Work updates (accept, start, resolve)
- Administrator dashboard and user management
- Reporting and analytics

- Audit trail and role-based access control

4.2 Out of Scope for MVP

- Full residence / accommodation management or allocation
- Payments or residence fee management
- General social / community platform
- Mandatory native mobile applications
- Complex predictive maintenance requiring large historical datasets
- Integration with every TUT system from day one
- Unbounded AI features without measurable operational value

5. Stakeholders

Stakeholder	Interest / Responsibility
Students	Report and track maintenance issues affecting their rooms and residence areas.
Maintenance Staff	Receive, perform, update and resolve maintenance work.
Residence Administrators	Manage users, requests, assignments, settings and reporting.
Residence Management	Monitor maintenance performance and recurring faults.
System Administrator	Maintain system configuration, security and operational controls.

6. User Roles & Access Model

FixFlow enforces the principle of least privilege: users only see and can perform actions necessary for their role. Administrative powers are never granted merely because a user is a staff member — role assignment is explicit and server-enforced.

6.1 Student

- Activate account and log in.
- Create maintenance reports.
- Upload evidence.
- View own reports and track request status.
- Add comments.
- Confirm resolution or dispute / reopen a resolution.

6.2 Maintenance Staff

- Log in and view assigned work.
- Accept jobs and start work.
- Update status and add notes.
- Upload completion evidence.
- Mark work as resolved.

6.3 Administrator

- Manage users and staff.
- Manage requests and assignments.
- Manage categories and priorities.
- Monitor overdue requests and escalations.
- View reports, analytics and audit logs.
- Configure system settings.

6.4 Role Summary

Role	Primary Responsibility	Key Permissions
Student	Report and track maintenance issues affecting their room / residence areas.	Create reports, view own reports, add evidence/comments, confirm or dispute resolution.
Maintenance Staff	Receive, inspect, update and resolve assigned maintenance work.	View assigned/eligible jobs, accept work, update status, add notes/evidence, mark complete.
Admin	Operate and oversee the entire system.	Manage users, staff, categories, priorities, assignments, escalations, reports, audit logs, settings.

7. Verified Student Access & Onboarding

Students do not freely create FixFlow accounts. Access is based on an authoritative Corridor Hill resident/student dataset, imported and controlled by an administrator.

7.1 Registration Flow

Step	System Behaviour
1. Student number	Student enters their TUT student number.
2. Resident validation	System checks whether the student number exists in the authorised Corridor Hill resident dataset and whether the record is active/eligible.
3. Account state	If eligible and not activated, continue to first-time account setup. If already activated, direct the student to normal login/recovery.
4. Secure password setup	Student creates a password meeting the configured security policy; stored only as a strong one-way hash.
5. Authentication	Future logins use student number + password, or an approved institutional identity provider if later integrated.
6. Session	System creates a secure authenticated session and applies the Student role.

7.2 Identity Verification Enhancement

A student number existing in a spreadsheet or database is not, by itself, sufficient proof of identity. Where an authoritative TUT identity service, institutional email verification, or another trusted identity mechanism is available, it should be used as an additional verification factor. Where unavailable, FixFlow relies on controlled resident-data validation plus rate limiting and an administrator-assisted recovery process.

7.3 Resident Data Handling

- Resident records are imported through a controlled admin process — the source dataset is never user-editable.
- Only the minimum personal data needed for the application is stored.
- Record status is tracked (active, inactive, checked-out, blocked) where operationally necessary.
- The resident import process is protected and administrative changes are logged.
- The resident list is never exposed through public APIs or error messages.

7.4 Maintenance Staff & Administrator Access

- Each staff member receives a unique staff identifier/number.
- Staff accounts are created or activated by an authorised administrator, never via public self-registration.
- Operational attributes are captured: discipline, team, availability and active/inactive status.
- Staff accounts require a secure password, with institutional email/OTP verification as an optional enhancement.
- RBAC ensures a staff account cannot reach administrative functions by altering client-side values.

8. Functional Requirements

ID	Requirement	Priority
FR-01	System shall validate a student's eligibility against the authorised Corridor Hill resident dataset.	Core
FR-02	System shall support first-time password setup for eligible students.	Core
FR-03	System shall authenticate returning users securely.	Core
FR-04	System shall enforce role-based access for students, maintenance staff and administrators.	Core
FR-05	Students shall create maintenance reports with structured information and optional evidence.	Core
FR-06	System shall generate a unique reference for every maintenance request.	Core
FR-07	Maintenance staff shall view and manage assigned work.	Core
FR-08	Administrators shall manage users, requests, assignments and system configuration.	Core
FR-09	System shall maintain an auditable history of important actions and status changes.	Core
FR-10	System shall notify relevant users of important workflow events.	Core
FR-11	System should detect or flag likely duplicate/related requests.	Enhancement
FR-12	System should provide operational analytics and recurring-fault insights.	Enhancement

9. Non-Functional Requirements

Area	Requirement
Security	Strong authentication, RBAC, validation, auditability and secure file handling.

Area	Requirement
Usability	Short, mobile-friendly and understandable student issue submission.
Performance	Common dashboard and submission actions respond promptly under expected residence load.
Availability	Monitoring, backups and documented recovery procedures.
Maintainability	Modular codebase, migrations, environment configuration, logging and API documentation.
Accessibility	Readable typography, keyboard accessibility, contrast, labels and responsive layouts.
Scalability	Architecture allows additional residences later without rewriting the core workflow.

10. User Stories

ID	User Story
US-01	As a student, I want to log in so that I can securely access maintenance services.
US-02	As a student, I want to report a maintenance problem so that the residence can address it.
US-03	As a student, I want to upload a photo so that maintenance staff can understand the problem.
US-04	As a student, I want to track my request so that I know whether it has been assigned or resolved.
US-05	As a student, I want to confirm or dispute a resolution so that incorrect resolutions can be reopened.
US-06	As maintenance staff, I want to see assigned jobs so that I know what work needs to be completed.
US-07	As maintenance staff, I want to update a job's status so that progress is visible.
US-08	As maintenance staff, I want to upload completion evidence so that there is proof of work.
US-09	As an administrator, I want to assign requests so that every request has a responsible person.
US-10	As an administrator, I want to monitor overdue requests so that work can be escalated.
US-11	As an administrator, I want to view analytics so that I can identify recurring problems.

11. Maintenance Workflow (State Machine)

Every report follows a clear, auditable state machine. This prevents requests from disappearing into a generic “submitted” bucket.

Status	Meaning	Typical Actor
Submitted	Issue has been captured and stored.	System / Student
Triaged	Issue has been classified and priority assessed.	Admin / Dispatcher
Assigned	A responsible maintenance staff member/team is identified.	Admin / Dispatcher
Accepted	Assigned staff member has acknowledged the job.	Maintenance
In Progress	Physical work/inspection is underway.	Maintenance

Status	Meaning	Typical Actor
Awaiting Parts / Access	Work cannot continue because of a documented dependency.	Maintenance
Resolved	Maintenance work is reported as completed, with evidence where applicable.	Maintenance
Verified / Closed	Student or authorised staff confirms the issue is resolved, or the system closes it per policy.	Student / Admin
Reopened	Issue persists or was incorrectly marked resolved.	Student / Admin

Linear flow: Submitted → Triaged → Assigned → Accepted → In Progress → Awaiting Parts/Access → Resolved → Verified/Closed, with a Reopened branch from Resolved/Closed back into the active workflow if the issue persists.

11.1 Student Report Form

- Maintenance category: plumbing, electrical, furniture, appliance, network, building, cleaning-related fault, safety, other.
- Specific location: building / area / floor / room or common-area location.
- Short issue title and plain-language description.
- Photo / video evidence where useful.
- Optional voice-to-text or guided prompts (future accessibility enhancement).
- Availability / access information where a technician needs entry.
- Automatic timestamp and unique issue/reference number.

12. Use Cases

UC-01 — Student Login

1. Student enters student number and password.
2. System validates credentials and account status.
3. System creates an authenticated session.
4. Student dashboard opens.

UC-02 — Submit Maintenance Request

5. Student selects category and location.
6. Student enters title and description.
7. Student uploads optional evidence.
8. Server validates information.
9. System creates request and unique reference number.
10. Status becomes SUBMITTED.
11. Student receives confirmation.

UC-03 — Assign Request

12. Admin opens request.
13. Reviews category, location and priority.
14. Selects maintenance staff.

15. System records assignment.
16. Status becomes ASSIGNED.
17. Staff receives notification.

UC-04 — Resolve Request

18. Staff opens assigned job.
19. Accepts job and changes status to IN PROGRESS.
20. Performs maintenance.
21. Adds notes / evidence.
22. Marks request RESOLVED.
23. Student is notified.

UC-05 — Confirm Resolution

24. Student reviews resolved request.
25. Student confirms or disputes.
26. Confirm → VERIFIED / CLOSED.
27. Dispute → REOPENED.

13. Differentiating Features

These features are designed to make FixFlow feel like a complete operational product rather than a CRUD complaint form. Admin retains override control in every case.

Feature	How It Works	Value
Smart Triage	Uses category, location, severity indicators and issue history to suggest priority and routing.	Reduces manual sorting and improves consistency.
Duplicate Detection	Flags potentially similar open reports for the same area/category before a new report becomes duplicate work.	Prevents repeated requests for the same fault.
Live Status Timeline	Students see a clean timeline of what happened to their request, without exposing internal/private notes.	Improves transparency; reduces “is anyone handling this?” questions.
Maintenance Work Queue	Staff see available/assigned work with priority, location, age, evidence and action buttons.	Makes the platform useful to technicians, not only students.
Escalation Engine	Automatically flags jobs approaching or exceeding configured service targets.	Makes overdue work visible before it is forgotten.
Resolution Evidence	Technician attaches completion notes/photos; student can confirm or reopen.	Creates accountability and a stronger audit trail.
Recurring Fault Intelligence	Aggregates historical issues by location/category to reveal repeated failures.	Turns maintenance data into residence insight.
Maintenance Heatmap	Visualises issue concentration by residence location, without exposing sensitive student data.	Helps management identify problematic areas.

Feature	How It Works	Value
Incident Mode	Admins can link many related reports to one master incident (e.g. building-wide outage).	Avoids treating one fault as 30 unrelated jobs.

14. Database Schema

A relational database is used because FixFlow contains structured relationships, workflow states, permissions, audit history and reporting requirements.

14.1 Design Rules

- Primary keys uniquely identify records.
- Foreign keys maintain relationships between entities.
- Student numbers and staff numbers are unique.
- Maintenance request reference numbers are unique.
- Status changes are preserved in request status history.
- Assignments preserve who was responsible and when.
- Evidence metadata is stored in the database while files are stored in protected object/file storage.
- Audit logs record sensitive operations.
- Database credentials use least privilege.
- User and resident data is minimised and access-controlled.

14.2 Core Tables

USERS

user_id PK
username UNIQUE
password_hash
role_id FK
email
phone
account_status
created_at
updated_at
last_login

ROLES

role_id PK
role_name UNIQUE
description

STUDENTS

student_id PK
user_id FK
student_number UNIQUE
first_name
last_name
room_id FK
resident_status
activation_status
created_at
updated_at

STAFF

staff_id PK
user_id FK
staff_number UNIQUE
first_name
last_name
discipline
team
availability_status
active_status
created_at
updated_at

RESIDENCES

residence_id PK
residence_name
address
status
created_at

LOCATIONS

location_id PK
residence_id FK
building
area

floor
room_number
location_type
description

MAINTENANCE_CATEGORIES

category_id PK
category_name
description
default_priority
active_status
created_at

MAINTENANCE_REQUESTS

request_id PK
reference_number UNIQUE
student_id FK
location_id FK
category_id FK
title
description
priority
status
access_information
submitted_at
triaged_at
resolved_at
closed_at
reopened_at
updated_at

REQUEST_STATUS_HISTORY

history_id PK
request_id FK
old_status
new_status
changed_by FK

change_reason
created_at

ASSIGNMENTS

assignment_id PK
request_id FK
staff_id FK
assigned_by FK
assigned_at
accepted_at
completed_at
assignment_status

ATTACHMENTS

attachment_id PK
request_id FK
uploaded_by FK
file_name
file_type
file_size
storage_path
attachment_type
uploaded_at

COMMENTS

comment_id PK
request_id FK
user_id FK
comment_text
comment_type
created_at

NOTIFICATIONS

notification_id PK
user_id FK
request_id FK
notification_type

title
message
is_read
created_at
read_at

AUDIT_LOGS

audit_id PK
user_id FK
action
entity_type
entity_id
old_value
new_value
ip_address
user_agent
created_at

INCIDENTS

incident_id PK
incident_reference UNIQUE
title
description
location_id FK
severity
status
created_by FK
created_at
resolved_at

REQUEST_INCIDENTS

request_id PK/FK
incident_id PK/FK

14.3 Suggested Data Values

Field	Suggested Values
roles.role_name	STUDENT / MAINTENANCE / ADMIN
location_type	ROOM / COMMON_AREA / KITCHEN / BATHROOM / CORRIDOR / OTHER
category_name	PLUMBING / ELECTRICAL / FURNITURE / APPLIANCE / NETWORK / BUILDING / CLEANING / SAFETY / OTHER
request.status	SUBMITTED / TRIAGED / ASSIGNED / ACCEPTED / IN_PROGRESS / AWAITING_PARTS_ACCESS / RESOLVED / VERIFIED_CLOSED / REOPENED
comment_type	STUDENT / INTERNAL / RESOLUTION

14.4 Entity Relationship Diagram — Text Schema

ROLES (1) —< USERS
 USERS (1) — (1) STUDENTS
 USERS (1) — (1) STAFF
 RESIDENCES (1) —< LOCATIONS
 LOCATIONS (1) —< MAINTENANCE_REQUESTS
 STUDENTS (1) —< MAINTENANCE_REQUESTS
 MAINTENANCE_CATEGORIES (1) —< MAINTENANCE_REQUESTS
 MAINTENANCE_REQUESTS (1) —< REQUEST_STATUS_HISTORY
 MAINTENANCE_REQUESTS (1) —< ASSIGNMENTS >— (1) STAFF
 MAINTENANCE_REQUESTS (1) —< ATTACHMENTS
 MAINTENANCE_REQUESTS (1) —< COMMENTS
 MAINTENANCE_REQUESTS (1) —< NOTIFICATIONS
 USERS (1) —< AUDIT_LOGS
 INCIDENTS (1) —< REQUEST_INCIDENTS >— (1) MAINTENANCE_REQUESTS

15. System Architecture

15.1 High-Level Architecture

FixFlow is delivered as a layered, web-first application:

Layer	Responsibility
Presentation	Responsive web application serving Student UI, Maintenance UI and Admin UI over HTTPS/API.
Application / API	Authentication, RBAC, validation, request workflow, assignment, reporting and audit logging.
Data Layer	Relational database holding users, requests, locations, staff, history, assignments and notifications.
Supporting Services	Secure file storage, notification delivery, monitoring, logging, backup and recovery.

A web-first responsive application is recommended because it serves students, technicians and administrators from one deployable system while remaining accessible on phones and computers. Client-side UX is never treated as an authoritative security boundary.

15.2 Student System Flow

- Open website → enter student number → resident validation.
- Not eligible → access denied.
- Eligible → first-time activation → create password → login.
- Student dashboard: Report Issue, My Requests, Request Details, Notifications.

15.3 Maintenance Staff Flow

- Login → staff dashboard → view assigned jobs → open job.
- Accept → In Progress → perform maintenance → add notes/evidence.
- Resolved → student verification → Closed.

15.4 Administrator Flow

Login → Admin Dashboard covering: Users, Students, Staff, Requests, Assignments, Categories, Priorities, Escalations, Reports, Analytics, Audit Logs, Settings.

15.5 Authentication Architecture

- User submits credentials via the login form.
- API server validates credentials — failure returns a generic error message.
- Success → secure session created.
- Role check routes the user: Student → Student UI, Staff → Staff UI, Admin → Admin UI.

15.6 Maintenance Request Architecture

28. Student completes the report form.
29. Server validates all fields — invalid submissions return field-level errors.
30. On success: request created, reference generated, evidence metadata stored, status set, audit entry created.
31. Status → SUBMITTED, then TRIAGE (category, priority, location).
32. Assignment routes the request to Maintenance Staff.

33. Workflow proceeds through resolution.
34. Student verification: Confirm → CLOSED; Dispute → REOPENED.

16. API Design

Area	Representative Endpoints
Authentication	POST /api/auth/activate POST /api/auth/login POST /api/auth/logout POST /api/auth/forgot-password POST /api/auth/reset-password
Student Requests	POST /api/requests GET /api/requests GET /api/requests/:id POST /api/requests/:id/comments POST /api/requests/:id/confirm POST /api/requests/:id/reopen
Maintenance	GET /api/staff/jobs GET /api/staff/jobs/:id POST /api/staff/jobs/:id/accept POST /api/staff/jobs/:id/start POST /api/staff/jobs/:id/resolve
Administration	GET /api/admin/users PATCH /api/admin/users/:id GET /api/admin/requests POST /api/admin/requests/:id/assign GET /api/admin/reports GET /api/admin/audit-logs

All endpoints require an authenticated session and are subject to server-side RBAC; the list above is representative, not exhaustive, and will be finalised during API specification (OpenAPI/Swagger) in System Design.

17. Security & Privacy

Security is treated as a core product requirement in FixFlow, not an optional enhancement.

Control	Implementation Direction
Authentication	Secure password hashing; protected sessions/tokens; account activation states; secure password recovery.
Authorisation	Server-side RBAC for Student, Maintenance and Admin; never rely on hidden UI controls alone.
Input validation	Validate and constrain all request fields, file metadata, identifiers and state transitions on the server.
Rate limiting	Limit repeated login, first-time activation, password reset and suspicious request attempts.
Account enumeration protection	Avoid revealing whether a student number is valid through overly specific error messages.
File security	Restrict file types/sizes, validate uploads, store outside executable paths, controlled access to evidence.
Audit logging	Record account changes, assignments, status changes, admin actions and security events.
Database security	Parameterised queries/ORM protections, least-privilege credentials, backups, restricted network access.
Transport security	HTTPS/TLS for production traffic; secure cookies; no credentials over insecure channels.
Secrets	Credentials/API keys kept in environment or secret-management facilities, never source code.

Control	Implementation Direction
Privacy	Minimum necessary personal information, restricted access, defined retention, aligned with TUT/residence and South African privacy requirements.
Operational recovery	Backups, restore testing, error handling and incident-response procedures.

Security target: design against broken access control, injection, authentication failures, insecure file handling, excessive data exposure and security misconfiguration — and test the implementation rather than merely claiming compliance.

17.1 Security Data Flow

Internet → HTTPS/TLS → Web Server → Authentication → API Server → Server-side RBAC → Database (least privilege) and File Storage (controlled access) in parallel → Audit Logs → Monitoring.

18. Notification & Communication Layer

- Student receives confirmation when a report is submitted.
- Student receives notifications when status changes or additional information is requested.
- Maintenance staff receive new/assigned job notifications.
- Admins receive escalation alerts and system/security alerts.
- In-app notifications are the primary system of record; email/SMS/push are optional delivery channels.
- Notification preferences are configurable where appropriate.
- Sensitive information is kept out of notification previews.

19. Dashboard Design

19.1 Student Dashboard

- Prominent “Report a Problem” call to action.
- My Requests summary: Open / In Progress / Resolved counts.
- Recent requests list with reference number and current status.

19.2 Maintenance Dashboard

- Assigned / In Progress / Awaiting Parts / Resolved counters.
- Prioritised job list with Priority, Location and Status columns.

19.3 Admin Dashboard

- Total, Open, Assigned, Overdue and Resolved request counts.
- Requests-by-category breakdown (Plumbing, Electrical, Network, Furniture, etc.).
- Recent / overdue requests requiring attention.

19.4 Suggested Web Pages

Role	Pages
Student	/login, /student/dashboard, /student/report, /student/requests, /student/requests/:id, /student/notifications, /student/profile

Role	Pages
Maintenance	/staff/dashboard, /staff/jobs, /staff/jobs/:id, /staff/profile
Admin	/admin/dashboard, /admin/users, /admin/students, /admin/staff, /admin/requests, /admin/assignments, /admin/categories, /admin/priorities, /admin/reports, /admin/audit-logs, /admin/settings

20. Requirements Traceability

Requirement	Database	UI / System Function
FR-01 Student validation	students	Account activation
FR-02 Password setup	users	Activation
FR-03 Login	users	Login
FR-04 RBAC	users, roles	Role-specific dashboards
FR-05 Report issue	maintenance_requests	Student report form
FR-06 Reference number	maintenance_requests	Request details
FR-07 Staff work	assignments	Staff dashboard
FR-08 Admin management	Multiple tables	Admin dashboard
FR-09 Audit history	audit_logs, request_status_history	Audit / history views
FR-10 Notifications	notifications	Notification centre
FR-11 Duplicate detection	maintenance_requests	Report / admin workflow
FR-12 Analytics	Request data and related tables	Admin analytics dashboard

21. Acceptance Criteria

ID	Acceptance Criterion
AC-01	Only eligible/authorised Corridor Hill residents can activate student accounts.
AC-02	A user can only access functions permitted by their role.
AC-03	A student can submit a maintenance request containing required category, location, title and description.
AC-04	Every submitted request receives a unique reference number.
AC-05	A student can view the status of their own requests.
AC-06	An administrator can assign a request to maintenance staff.
AC-07	Assigned staff can accept, start and resolve work.
AC-08	Status transitions are recorded with actor and timestamp.
AC-09	Students receive notifications for relevant workflow events.
AC-10	A student can confirm or reopen a resolution.

ID	Acceptance Criterion
AC-11	Sensitive administrative actions are recorded in audit logs.
AC-12	Evidence uploads are validated and access-controlled.

22. Testing Strategy

- Unit testing of validation and workflow functions.
- Integration testing of frontend, API and database.
- Authentication testing.
- Role / permission testing.
- Maintenance workflow / state-transition testing.
- File-upload security testing.
- Notification testing.
- Audit logging verification.
- Usability testing on mobile and desktop.
- Accessibility testing.
- Performance testing under expected residence load.
- Backup and restore testing.
- Security testing against broken access control, injection, authentication failures, insecure file handling, excessive data exposure and security misconfiguration.

23. Risk Register

Risk	Impact	Mitigation
Resident spreadsheet is outdated	Unauthorised or blocked students may be incorrectly handled.	Define source-of-truth ownership, import validation, timestamps and admin reconciliation.
Student number alone treated as identity proof	Account takeover / unauthorised activation risk.	Add trusted identity/email verification where available; rate limit activation.
Maintenance staff do not update statuses	Dashboard becomes inaccurate.	Make workflow fast and mobile-friendly; use reminders/escalations.
Duplicate reports	Wasted maintenance effort.	Duplicate/related-issue detection and master incident workflow.
Poor evidence quality	Technicians cannot diagnose issues remotely.	Guided reporting fields and optional photo evidence.
Overbuilding for the challenge	Core system remains unfinished.	Freeze MVP scope; add enhancements only after core workflow is stable.
Sensitive data exposure	Privacy and security incident.	Least privilege, secure storage, access controls, logging, testing and data minimisation.

24. Analytics & Success Metrics

FixFlow proves its impact with measurable operational metrics rather than screenshots alone:

- Number of maintenance requests submitted.
- Percentage of requests successfully assigned.
- Average time from submission to triage.
- Average time from assignment to first staff action.
- Average time to resolution.
- Number / percentage of overdue requests.
- Reopened-request rate.
- Recurring faults by location / category.
- Requests grouped under major incidents.
- Student confirmation / feedback after resolution.

These metrics feed the admin analytics dashboard and the demonstration story: FixFlow does not merely record complaints — it creates measurable operational visibility.

25. SDLC Roadmap

Phase	Outputs for FixFlow
1. Planning	Problem statement, objectives, stakeholders, assumptions, constraints, project scope, initial risk register.
2. Requirements Analysis	Functional requirements, non-functional requirements, user stories, acceptance criteria, role/permission matrix.
3. System Design	Architecture, database ERD, API design, authentication flow, state machine, security design, wireframes/UI system.
4. Development	Frontend, backend/API, database, authentication, file handling, notifications, dashboards and integrations.
5. Testing	Unit tests, integration tests, role/permission tests, authentication tests, file-upload tests, usability tests, security testing.
6. Deployment	Production environment, database migrations, HTTPS, secrets, backups, monitoring and release checklist.
7. Maintenance	Bug fixes, monitoring, user feedback, performance improvements, security updates and measured feature iterations.

Recommended build order: requirements → data model → authentication/authorisation → maintenance workflow → dashboards → notifications → intelligence → hardening/testing → deployment.

26. MVP Delivery Sequence

Milestone	Deliverable
M1 — Foundation	Requirements, user stories, architecture, database schema, repository structure and development environment.

Milestone	Deliverable
M2 — Identity	Resident validation, first-time activation, login, staff onboarding and RBAC.
M3 — Student Reporting	Issue form, evidence upload, reference number, report history and status timeline.
M4 — Maintenance Operations	Staff queue, assignment, accept/start/complete workflow, notes and evidence.
M5 — Administration	Admin dashboard, user management, assignment controls, categories, priorities and audit logs.
M6 — Intelligence	Duplicate detection, incident linking, escalation rules and operational analytics.
M7 — Hardening	Security testing, role testing, error handling, backups, monitoring, accessibility and performance checks.
M8 — Demonstration	End-to-end scenario, metrics, architecture explanation, security story and polished UX.

27. Final Deliverables Checklist

- Complete Software Requirements Specification (SRS)
- Functional requirements
- Non-functional requirements
- User stories
- Use cases
- Maintenance workflow / state machine
- Database schema
- Entity relationships / ERD text schema
- High-level system architecture
- Authentication architecture
- Maintenance request architecture
- API endpoint specification
- Security architecture
- Dashboard design
- Requirements traceability
- Acceptance criteria
- Testing checklist
- Risk register
- MVP build sequence

28. Conclusion

FixFlow is structured as a secure, residence-specific web platform that connects verified students, maintenance staff and administrators through a complete maintenance lifecycle.

The SRS, database model and system design in this document are intended to remain traceable: each requirement maps to data, system behaviour, user interface functionality and a testable acceptance criterion.

The MVP remains focused on maintenance issue management while keeping the architecture extensible for future residences and carefully selected enhancements. This document is the baseline for stakeholder sign-off and the start of System Design implementation.